COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	08/04/2026	**DATE SUBMITTED:**	08/04/2026
NAME:	Cabug, Jeffordniel H.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 5.1 QUEUES

1. PROCEDURE OUTPUT

ILO A:

CODE:

ILOA > main.cpp > main()

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    // create an object
    std::queue<char> myQ;

    myQ.push('J');
    myQ.push('E');
    myQ.push('F');
    myQ.push('F');
    myQ.push('O');
    myQ.push('R');
    myQ.push('D');
    myQ.push('N');
    myQ.push('I');
    myQ.push('E');
    myQ.push('L');
    display(myQ);

    // TRY ALL MEMBER FUNCTIONS:
    // empty
    std::cout<<"isEmpty: "<<myQ.empty()<<std::endl;
    // size
    std::cout<<"Size: "<< myQ.size()<<std::endl;
    // front
    std::cout<<"Front: "<< myQ.front()<<std::endl;
    // back
    std::cout<<"Back: "<< myQ.back()<<std::endl;
    // pop
    std::cout<<"Popping: "<< myQ.front()<<std::endl;
    myQ.pop();
    // display
    display(myQ);
    // push
    std::cout<<"Pushing: "<< myQ.back()<<std::endl;
    myQ.push('J');
    // display
    display(myQ);

    return 0;
}
void display(std::queue<char> copyQ){
    //creates a copy of the queue that was passed
    std::queue<char> c = copyQ;

    while (!c.empty())
    {
        std::cout<<" "<<c.front();
        c.pop();
    }
    std::cout<<"\n";

}
```

OUTPUT:

```
[Running] cd "c:\Users\TIPQC\Documents\CPE21S3_C
J E F F O R D N I E L
isEmpty: 0
Size: 11
Front: J
Back: L
Popping: J
E F F O R D N I E L
Pushing: L
E F F O R D N I E L J

[Done] exited with code=0 in 0.607 seconds
```

OBSERVATION:

The code shows queue STL. It adds the letters of the name into the queue, displays them, checks if the queue is empty, shows its size, front, and back elements, then removes the first letter using pop() and adds it back using push(). The output proves that a queue follows the First In, First Out (FIFO) rule, where the first element added is the first one removed, and new elements are always added at the end.

ILO B:

CODE:

MAIN.CPP

ILOB > main.cpp > main()

```cpp
#include <iostream>
#include "queueLL.h"

int main (){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    //enqueue
    enqueue('J', &front, &back);
    std::cout<< front->data << " " << back->data <<std::endl;
    enqueue('E', &front, &back);
    std::cout<< front->data << " " << back->data <<std::endl;
    enqueue('P', &front, &back);
    std::cout<< front->data << " " << back->data <<std::endl;

    //display JEP
    displayAll(front);

    //dequeue
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data <<std::endl;
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data <<std::endl;
    dequeue(&front, &back);

    //display Empty
    displayAll(front);

    //front
    enqueue('J', &front, &back);
    getFront(front);
    //back
    enqueue('E', &front, &back);
    getBack(back);
    std::cout<<front->data<<" "<<back-> data <<std::endl;

    // display ALL J and E
    displayAll(front);

    return 0;
}
```

QUEUELL.H

ILOB > C queueLLh > displayAll<T>(qNode<T>*)

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H


// create a class the node
template <typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

//creates the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

// enqueue operation
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    // create the new node
    qNode<T>* newNode = new_node(newData);

    //inserting into an empty queue
    if((*frontPtr)==nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }
    //inserting an item into a non empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //add a isempty function here

    //create a temporary node
    qNode<T>* deleteNode = nullptr; // temp variable
    deleteNode = (*frontPtr);

```

```cpp

    //check if the queue has only 1 node
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }
    // delete the node
    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;

}

//check the front
template<typename T>
void getFront(qNode<T>* frontPtr){
    if (frontPtr == nullptr){
        std::cout<<"Queue is empty!"<<std::endl;
        return;
    }

    std::cout<<"The current front: "<<frontPtr->data<<std::endl;

}

// check the back
//check the front
template<typename T>
void getBack(qNode<T>* backPtr){
    if (backPtr == nullptr){
        std::cout<<"Queue is empty!"<<std::endl;
        return;
    }

    std::cout<<"The current back: "<<backPtr->data<<std::endl;

}
//display all
template <typename T>
void displayAll(qNode<T>* frontPtr){
    if(frontPtr == nullptr){
        std::cout<<"The queue is empty!"<<std::endl;
        return;
    }
    std::cout<<"The current Queue: ";
    while(frontPtr != nullptr){
        std::cout<<frontPtr->data<<" ";
        frontPtr = frontPtr->next;
    }
    std::cout<<"\n";

}
#endif
```

OUTPUT:

```
[Running] cd "c:\Users\TIPQC\Documents\CPE21S3_C
J J
J E
J P
The current Queue: J E P
E P
P P
The queue is empty!
The current front: J
The current back: E
J E
The current Queue: J E

[Done] exited with code=0 in 0.615 seconds
```

OBSERVATION:

It uses two pointers, front and back, to keep track of the first and last elements in the queue. In the main.cpp file, the program inserts the letters J, E, and P using the **enqueue()** function, displays the front and back values after each insertion, and prints all the elements in the queue using **displayAll().** It then removes elements one by one using the **dequeue()** function and displays the updated queue. The **queueLL.h** file contains the node class and the functions for creating a new node, adding elements (enqueue), removing elements (dequeue), checking the front (getFront) and back (getBack) elements, and displaying all queue contents. The program also checks if the queue is empty before performing operations, which helps prevent errors. This code shows the queue operations and shows how a linked list can be used to implement a dynamic queue.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **AI overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Concepts Concept**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **Using AI to explain the STL briefly and how it works.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.